

Implementation Phases

Eight phases, ordered by dependency. Each has a Definition of Done that must be fully checked before moving on. If any item fails, fix it in place rather than carrying debt forward — every later phase assumes the earlier ones are solid.

Phase 1 — Schemas & Seed

Covers: Mongoose models for `User`, `Lead`, `Note`, `Activity`, plus a repeatable seed script.

Definition of Done

- All four model files exist under `src/models/` and load without errors when the server starts
 - `Lead` has indexes on `{ status: 1, followUpDate: 1 }` and `{ createdAt: -1 }`
 - `Note` and `Activity` have indexes on `{ leadId: 1, createdAt: -1 }`
 - `Note` and `Activity` do not have an `updatedAt` field (they are immutable)
 - Every model's `toJSON` transform renames `_id` to `id`, strips `__v`, and strips `passwordHash` (User only)
 - `npm run seed` completes without error and prints a summary
 - After seeding: `db.leads.countDocuments()` returns 12, `db.users.countDocuments()` returns 1
 - Deleting a lead in `mongosh` cascades to its notes and activity (count returns 0)
 - A temporary debug route proves `toJSON` strips `passwordHash` and `_id`
 - `.env` is gitignored and does not appear in `git status`
-

Phase 2 — Authentication

Covers: `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`, `requireAuth` middleware, cookie and CORS configuration.

Definition of Done

- `bcryptjs` hashes with cost factor 12
- Login with correct credentials returns 200 and sets an `auth` cookie

- Login with wrong password returns 401 with the same message as an unknown email
 - Login with an unknown email returns 401 with the same message as a wrong password
 - The auth cookie has `HttpOnly`, `Secure`, `SameSite=None`, `Path=/`, `Max-Age=604800`
 - The JWT payload contains `sub` (user id) and an expiry of 7 days
 - `requireAuth` rejects missing cookies with 401
 - `requireAuth` rejects malformed tokens with 401
 - `requireAuth` rejects expired tokens with 401
 - `GET /api/auth/me` returns `{ id, email, name }` — no `passwordHash`, no `_id`
 - Logout returns 204 and clears the cookie
 - After logout, `GET /api/auth/me` returns 401
 - Login is rate-limited: 11 requests in 15 minutes from the same IP returns 429 on the 11th
 - Cookies are not readable from `document.cookie` in the browser (verify in DevTools)
 - CORS allowlist uses an explicit origin, never `*`
 - Verified end-to-end in Postman: login → cookie set → `/me` returns 200 → logout → `/me` returns 401
-

Phase 3 — Core API

Covers: All CRUD routes for leads, notes, and activity.

Definition of Done

- `GET /api/leads` returns an empty list initially, then the seeded 12
- `GET /api/leads?status=new` filters correctly
- `GET /api/leads?search=thandi` matches name, email, and company
- `GET /api/leads?limit=5&offset=5` returns the correct page with correct `total`
- `POST /api/leads` creates a lead and returns 201
- `POST /api/leads` with an invalid email returns 400 with `fields.email`
- `POST /api/leads` with a missing required field returns 400 with the field named
- `POST /api/leads` with `status: "banana"` returns 400
- `GET /api/leads/:id` returns the lead with `notes` and `activity` arrays
- `GET /api/leads/:invalid-id` returns 400, not 500

- GET /api/leads/:missing-id returns 404
 - PATCH /api/leads/:id with { status: "contacted" } sets lastContactedAt automatically
 - PATCH /api/leads/:id with an explicit lastContactedAt does not override it
 - PATCH /api/leads/:id with an unknown field returns 400
 - DELETE /api/leads/:id returns 204
 - After delete, GET /api/leads/:id returns 404
 - After delete, the lead's notes and activity are gone from the database
 - POST /api/leads/:id/notes creates a note and returns 201
 - POST /api/leads/:id/notes with whitespace-only body returns 400
 - GET /api/leads/:id/notes returns notes newest-first
 - GET /api/leads/:id/activity returns entries newest-first
 - Every mutation creates the correct activity entry with the correct type and meta
 - Every route returns 401 without a valid cookie
 - No error response leaks a stack trace or a MongoDB error message
 - Verified end-to-end in Postman: create → read → update → add note → list activity → delete
-

Phase 4 – Public Capture

Covers: POST /api/capture, the honeypot, rate limiting, and the "message becomes a note" rule.

Definition of Done

- POST /api/capture with valid data returns 200 and creates a lead with status: "new"
- The created lead's source matches what was submitted
- The submitted message is stored as a Note with source: "capture" and authorId: null, not as a field on the Lead
- The lead_created activity has actorId: null
- POST /api/capture with a non-empty botField returns 200 and does not create a lead
- The response body is byte-identical whether a lead was created or the honeypot triggered
- The response does not contain the new lead's id
- The response does not reveal whether the email already exists in the system

- Submitting 6 times in an hour from the same IP returns 429 on the 6th
 - The endpoint works without any cookie
 - Validation errors return 400 with the same `fields` shape as admin creation
 - Verified in Postman: legitimate submission creates a lead; bot submission does not; rate limit fires
-

Phase 5 – Realtime (SSE)

Covers: `GET /api/events`, the event emitter, heartbeat, and connection registry.

Definition of Done

- `GET /api/events` returns `Content-Type: text/event-stream`
 - `GET /api/events` requires authentication (returns 401 without a cookie)
 - An open `curl -N` connection receives a `: ping` heartbeat at least every 25 seconds
 - Creating a lead via `POST /api/leads` emits a `lead.created` event with the full lead payload
 - Updating a lead emits a `lead.updated` event
 - Deleting a lead emits a `lead.deleted` event with the `leadId`
 - Adding a note emits both `activity.created` and `lead.updated` events
 - Submitting via `POST /api/capture` emits a `lead.created` event
 - The 6th concurrent SSE connection receives 429
 - Closing the client connection removes the response object from the server registry (no memory leak – verify via a log statement on `close`)
 - Events continue to fire after a client disconnects and reconnects
 - Verified with two terminals: `curl -N` in one, `curl -X POST` in the other, event appears in the first
-

Phase 6 – Frontend

Covers: Every page, the API client, TanStack Query, SSE subscription, optimistic updates, and theming.

Definition of Done

- The Tailwind config and `index.css` tokens match `docs/brand.html` exactly

- Login screen presents email and password fields, submits, and handles both success and failure
 - On failed login, the email field is not cleared and the error is shown inline
 - After successful login, the app routes to the dashboard
 - Dashboard shows leads as cards below 768px and as a table at 768px and above
 - Status filter and search both work and update the list
 - Follow-up dates are visually emphasised when due today or overdue
 - Lead detail screen shows metadata, status controls, follow-up picker, notes, and activity, in that order
 - Adding a note appears immediately in the timeline without a page refresh
 - Changing status updates the UI immediately and reverts on server error
 - Deleting a lead prompts for confirmation
 - Creating a lead from the dashboard works and appears immediately
 - The public capture form exists at a route that works without a session
 - Submitting the capture form succeeds and shows a confirmation
 - Opening the dashboard in two tabs and submitting the capture form in a third makes the new lead appear in both dashboard tabs without a refresh (this is the SSE test)
 - The realtime connection indicator shows connected/disconnected correctly
 - Killing the backend (or sleeping it) shows the disconnected indicator, and it recovers automatically when the backend returns
 - Light/dark toggle works on every screen and persists across reloads
 - First visit with no saved preference respects the OS setting
 - Every form input has an associated `<label>`
 - Every interactive element is reachable by Tab and shows a visible focus ring
 - Layout at 375px has no horizontal scroll
 - No component uses a banned Tailwind class from `no-slop.md`
 - Verified manually on the deployed URL, not just locally
-

Phase 7 — Tests & Polish

Covers: Automated tests, empty states, loading skeletons, error toasts, accessibility, and mobile.

Definition of Done

- `npm test` in `server/` runs a suite of at least 10 tests

- Tests cover: auth required, create lead, update status, add note, delete cascade, reject invalid input, reject unauthenticated read, honeypot, rate limit, invalid ObjectId
- Tests pass without a live Atlas connection (use `mongodb-memory-server` or a separate test database)
- Every list view has a defined empty state
- Every async view has a loading skeleton (not a spinner)
- Every mutation has an error toast on failure
- Delete confirmation modal appears and functions
- Lighthouse accessibility score ≥ 95 on the deployed URL
- Lighthouse SEO score ≥ 90
- Lighthouse performance score ≥ 90
- No console errors on any screen
- No console warnings from React about keys, controlled inputs, or missing dependencies
- The 404 route renders correctly